



Reproducible Testing and Secure Delivery for a Sailing Team MQTT Web Application

Ettore Mugisha Cirillo (emcir@kth.se) Juozas Skarbalius (juozas@kth.se)

KTH Royal Institute of Technology — DD2482, 2026

github.com/EccirilloM/kth_project_devops

1 Problem and scope

This project builds on an existing Angular web application developed by the Polimi Sailing Team to display boat telemetry and send commands through an MQTT broker. Verifying its interaction with the onboard system is difficult when the boat is unavailable. Our goal is to test changes without relying on the physical hardware and automate the deployment of the checked application.

We simplified the application to retain sign-in, a telemetry dashboard, a map and recording controls for operators. A simulator publishes synthetic telemetry and responds to recording commands, allowing us to exercise these features through a real broker.

Using this setup, we bring automated testing, infrastructure configuration and deployment together in a reproducible workflow for checking and releasing changes to the application.

2 Architecture and interactions

The application has three main components: the frontend, an MQTT broker and the boat simulator. After loading the frontend, the browser connects directly to the broker over secure WebSockets to receive telemetry and send commands. Through the same broker, the simulator publishes data in the format expected by the application and responds to recording commands.

When users sign in, the broker checks their credentials and applies the permissions assigned to their account: guests can read telemetry, while operators can also send recording commands. The frontend uses a public mapping of usernames to roles to display the appropriate controls, while the broker enforces access to the MQTT topics. Passwords are entered at sign-in and kept separate from the public connection settings.

For automated testing, Terraform creates the temporary Docker environment shown in Figure 1, containing the frontend server, broker and simulator. Playwright then exercises the application through a browser using real network connections. Docker also supplies the development and testing tools, which can be run locally through the provided Compose commands.

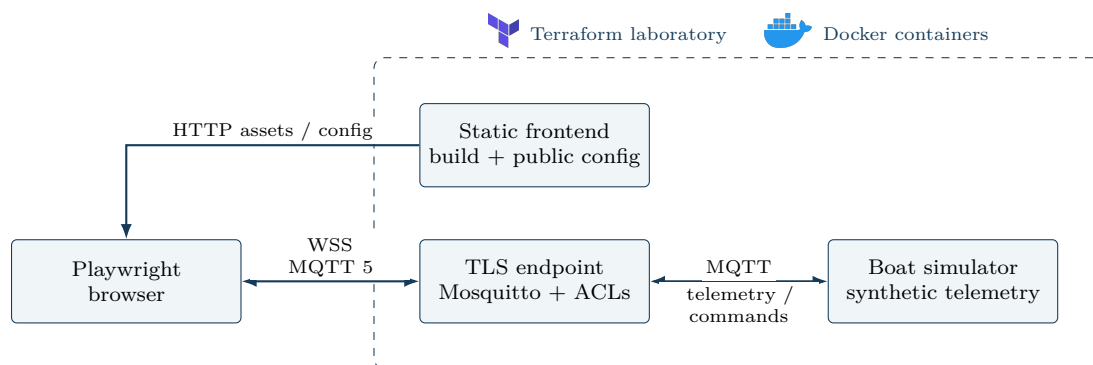


Figure 1: Temporary integration environment. Terraform manages the components inside the dashed boundary, and Playwright tests their interaction through the browser.

For the public demo, a separate Terraform configuration deploys the three components to DigitalOcean App Platform using images stored in DigitalOcean Container Registry (DOCR). Users can open the frontend there or on GitHub Pages; both connect to the same demo broker and display the simulator’s telemetry, including its position on a map supplied by OpenStreetMap tiles. This demo remains available while automated checks run in their own temporary environment.

3 Continuous integration and security

Whenever changes are pushed to the project branches or submitted through a pull request to `dev` or `main`, GitHub Actions runs the automated checks. The frontend, simulator and secret scan are checked in parallel, providing early feedback before testing the components together. These checks use containerized tools and the dependencies recorded in the npm lockfiles to keep local and CI runs consistent.

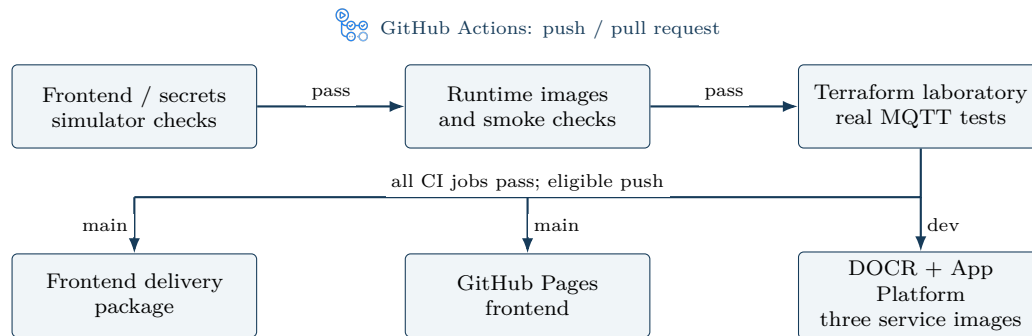


Figure 2: All CI stages must pass before delivery. Publication reuses their outputs; branch conditions select the deployment destination.

As shown in Figure 2, the pipeline progresses from checking the individual components to testing the application through a real MQTT connection. Along the way, it verifies that the frontend builds and starts correctly, checks the simulator’s behaviour and message formats, and validates the workflow and infrastructure configuration. Deployment can proceed only when all five CI jobs pass.

The quality and security checks follow the policies summarised in the table below. Dependency vulnerabilities rated high or critical block publication, while lower-severity findings remain visible for review. The pipeline also stops if a scanner cannot complete, so an unavailable check cannot be mistaken for a successful result.

Control	Purpose and blocking policy
ESLint and type checks	Detect inconsistent or invalid code; errors and lint warnings block.
Build and tests	Reject changes that fail compilation or automated verification.
Gitleaks	Detect exposed credentials in history and source; findings or scanner errors block.
Dependency audit	Report known dependency vulnerabilities, including development tools; high/critical findings and scan errors block.

To verify that these controls work, we include deliberate lint violations and nonfunctional test tokens in isolated checks. The checks must detect these examples for the demonstration to pass, confirming that the pipeline responds to known problems.

4 Delivery and deployment

Once all CI checks pass, the delivery workflow uses their outputs to publish the application. Changes on `main` publish the frontend to GitHub Pages and make its build available for download, while changes on `dev` update the demonstration on DigitalOcean App Platform.

The frontend is built once and reused throughout testing and delivery. A SHA-256 checksum manifest checks that the compiled files remain unchanged between these stages. Connection settings, such as the broker address, are supplied separately at runtime, allowing the same build to run in different environments.

For DigitalOcean, the workflow uploads the tested frontend, broker and simulator images to DigitalOcean Container Registry. Each release receives unique image tags, which Terraform uses to update the three components together in one infrastructure plan. The workflow checks the image digests before and after deployment, then verifies that the active application uses the expected images and frontend configuration.

Terraform keeps track of the deployed resources through a state file stored in a private, versioned DigitalOcean Spaces bucket. Locking prevents simultaneous changes to this state, while versioning retains earlier copies for recovery. Before updating the application, the workflow checks that the necessary configuration is available and stops if the plan would delete, replace or unexpectedly recreate resources. Initial setup and recovery are handled explicitly by an operator.

Deployment credentials are available only to the jobs that need them, and passwords, private keys and Terraform state are kept outside version control.

5 Verification and reproducibility

Successful workflow runs completed both deployment paths: GitHub Pages from `main` and DigitalOcean App Platform from `dev`. In each case, all five CI jobs passed before publication. We then checked the deployed application in the browser, confirming that users could sign in, receive telemetry and use the features available to their role. Workflow results are available in the repository’s Actions tab.

Area	Verification performed
Application and code quality	Type checking, linting, dependency and secret scans, unit tests, frontend startup and container checks.
MQTT integration	Telemetry updates, recording commands, malformed messages, stale data, recovery after a broker restart and rejection of guest commands.
Test infrastructure	Creation and cleanup of the temporary Docker environment, including repeated Terraform application to check idempotence.
Public application	Guest sign-in and changing telemetry on Pages; operator recording controls and map display on App Platform.

The integration tests exercise permissions through the broker itself: a guest command must be rejected without reaching the simulator, while an operator can send a command on the same topic successfully. This verifies access control alongside the application’s normal communication flow.

For the temporary Docker environment, the workflow applies the Terraform configuration again and checks that the resources remain unchanged. Test reports and diagnostic logs are saved before cleanup, including when a check fails, so that problems can be investigated afterward. The README provides the commands needed to reproduce the automated checks locally.

6 Discussion and collaboration

Design choices. We chose Terraform to describe the infrastructure in code and inspect planned changes before applying them. We paired it with Docker to provide a common environment for running the application and its checks. Since the frontend is served over HTTPS, we use secure WebSockets for its connection to the MQTT broker. Keeping connection settings separate from the frontend build then allows us to deploy the same tested files in different environments.

During development, Docker Hub download failures repeatedly interrupted the pipeline. We addressed these failures by using Google’s public image cache and retrying temporary download errors a limited number of times. These episodes showed us how the availability of external services can affect an otherwise working pipeline.

Scope and limitations. The simulator lets us test communication and application behaviour while the boat is unavailable. Testing with the boat and its operational broker would help assess how well these results transfer to the onboard environment. Our browser tests currently use Chromium, while the security checks cover exposed credentials and known npm dependency vulnerabilities. Building on this setup, future improvements could include testing with additional browsers and scanning operating-system packages in the container images. The public demo is intended to remain available for course assessment.

Collaboration. Juozas contributed the simulator and initial DigitalOcean infrastructure and deployment setup. Ettore adapted the frontend, developed the quality and security checks, and integrated and refined the CI/CD workflow and deployment configuration. We used shared MQTT message formats to coordinate the frontend and simulator, combining changes through personal branches, `dev` and `main`.

Project resources

[1] Source code and setup instructions: https://github.com/EccirilloM/kth_project_devops.

[2] Online application: https://eccirillom.github.io/kth_project_devops/.